

# Arm Robots with ROS: ALB5

M. J. Khojasteh

September 2026

## 1 Objective

In this experiment, students will become familiar with the AL5B Robotic Arm, a five-degree-of-freedom (5DOF) robot. The arm will be controlled using the Robot Operating System (ROS) by integrating a third-party package that connects the SSC-32U board to ROS.

Upon completion of this lab, students should be able to:

1. Utilize the ROS publisher-subscriber model to send commands to the robotic arm's joints.
2. Gain an understanding of the Unified Robot Description Format (URDF).
3. Develop a basic pick-and-place program.

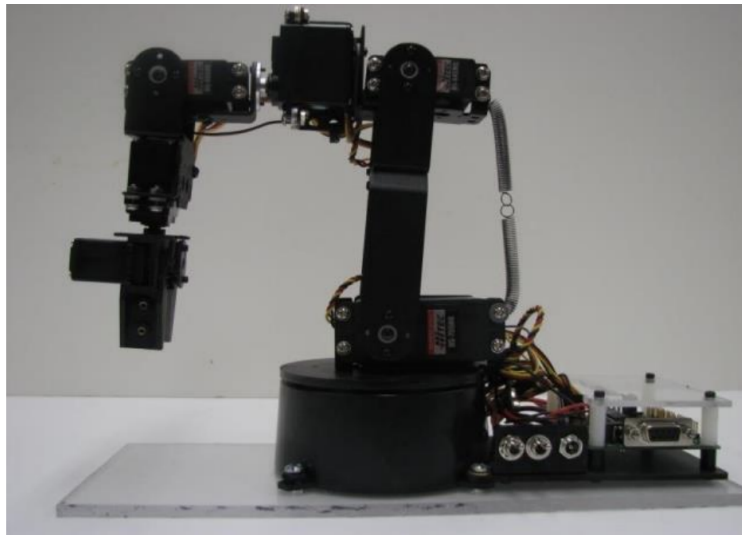


Figure 1: AL5B Robotic Arm

## 2 Pre-Lab

There is no pre lab for this lab exercise.

### 3 Introduction

1. **AL5B Robotic Arm Platform:** The AL5B robotic arm is a five-degree-of-freedom (5-DOF) robot equipped with seven servo motors. Each motor provides one degree of freedom to specific joints of the arm, including the base, shoulder, elbow, wrist, wrist twist, and gripper, as well as an elbow twist (which is excluded in this lab). These motors allow movement within a range of  $\pm 60^\circ$ .
2. **Servo motor & SSC32 servo controller:** The servo motors shown in Figure 2 require a positive pulse ranging from 0.5 to 2.5 milliseconds, followed by an off-time between 5 and 20 milliseconds. The pulse duration (on-time) determines the position of the servo's output shaft, while the off-time allows the servo to rest. As illustrated in Figure 2, the position of the output shaft is directly proportional to the pulse width, enabling precise control between the full left and full right positions. Multiple servo motors can be efficiently controlled using the SSC-32 servo controller board, as implemented in the AL5B robotic arm.
3. **Connecting the robot arm:** The servos are powered by a 5V adapter, while the SSC-32 logic is supplied with 5V from a separate power source. Ensure that the VL = VS1 jumper is removed. Then, connect the micro USB cable to the SSC-32 board as shown below.

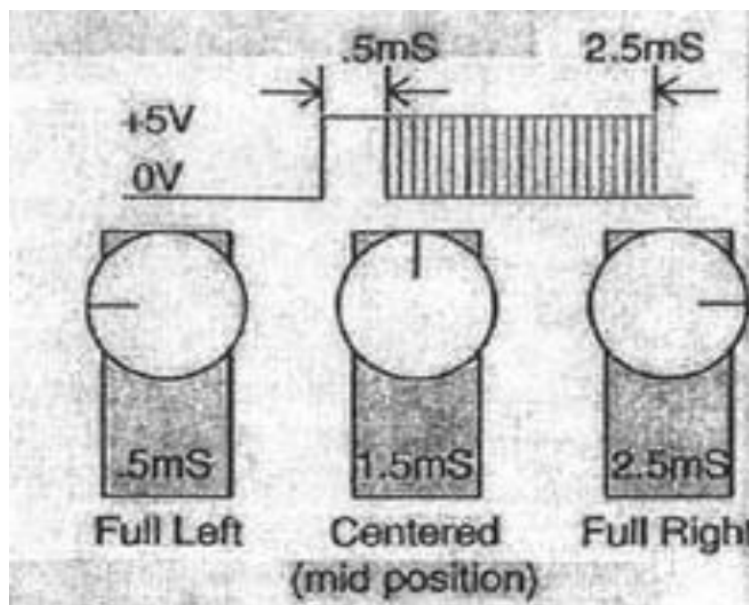


Figure 2: Servo positions based on provided PWM signals.

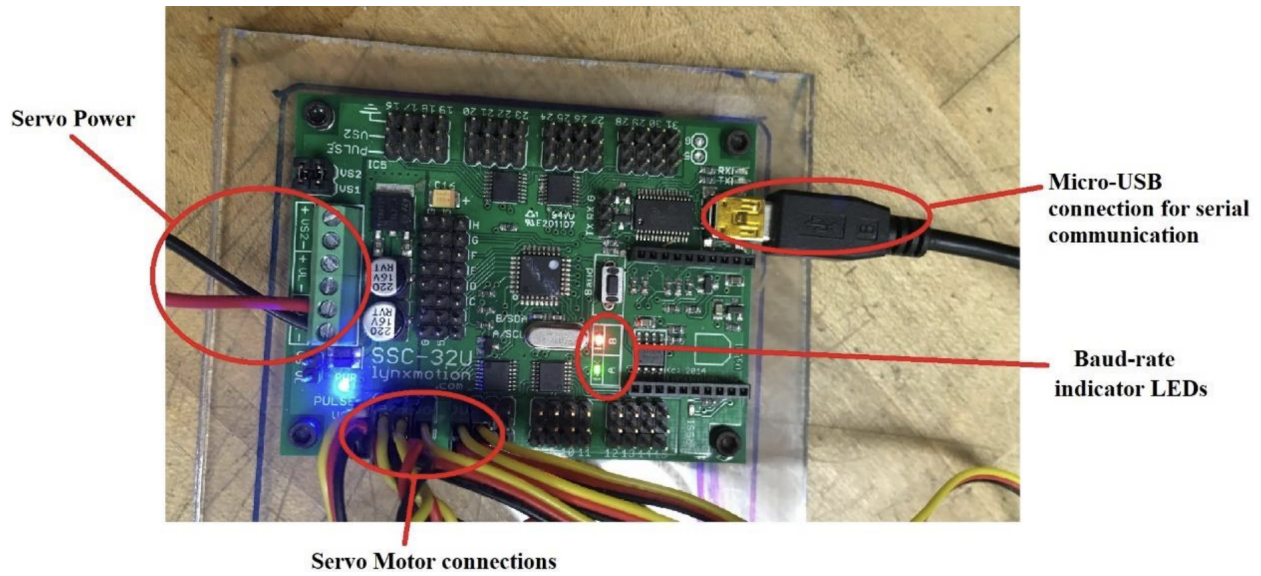


Figure 3: Arm Robot connection

## 4 Theory

### Frame Transformations

Determining the position and orientation of a robotic arm's joints is a fundamental task in robotics. To do this, an initial reference frame—known as the Universal Frame (U)—must first be established. This frame is defined using Cartesian coordinates and consists of three orthogonal unit vectors:  $x$ ,  $y$ , and  $z$ . Once the universal frame is set, additional coordinate frames can be assigned to each joint of the robot. In this case, five such fixed reference frames are used, excluding the universal frame (U) and the tool frame (T). Each of these joint frames is also expressed in Cartesian coordinates, using three orthogonal unit vectors:  $n$ ,  $o$ , and  $a$ . These vectors are defined relative to the universal frame's  $x$ ,  $y$ , and  $z$  axes (see Figure 1). The origin of each joint frame is defined by a position vector  $P$ , which also has components in the  $x$ ,  $y$ , and  $z$  directions relative to the universal frame (Figure 2). As shown in Figure 2, the vectors  $n$ ,  $o$ ,  $a$ , and  $P$  are combined into a single matrix using homogeneous coordinates. This matrix format maintains square dimensionality, which allows for direct application of transformation operations. When describing the position and orientation of any frame, it is always defined relative to a preceding frame or the universal frame. These relationships are captured through frame transformations, which consist of pure translations and/or rotations relative to the previous frame. By analyzing the geometry between consecutive frames, the necessary transformations can be identified (Figures 3–6). These transformations are applied to each frame by pre-multiplying the corresponding matrices, following the specific sequence of rotations and translations.

$$\vec{n} = \begin{bmatrix} n_x \\ n_y \\ n_z \end{bmatrix} \quad \vec{o} = \begin{bmatrix} o_x \\ o_y \\ o_z \end{bmatrix} \quad \vec{a} = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}$$

Figure 4 - Fixed Reference Frame Unit Directional Vectors

$$F = \begin{bmatrix} n_x & o_x & a_x & P_x \\ n_y & o_y & a_y & P_y \\ n_z & o_z & a_z & P_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 5 - Fixed-Reference Frame Homogeneous Coordinates

$$\begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 6 - Pure Translation

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\phi) & -\sin(\phi) & 0 \\ 0 & \sin(\phi) & \cos(\phi) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 7 - Pure Rotation (X-Axis)

$$\begin{bmatrix} \cos(\phi) & 0 & \sin(\phi) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\phi) & 0 & \cos(\phi) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 8 - Pure Rotation (Y-Axis)

$$\begin{bmatrix} \cos(\phi) & -\sin(\phi) & 0 & 0 \\ \sin(\phi) & \cos(\phi) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 9 - Pure Rotation (Z-Axis)

## 5 Lab Instructions

Before we can start with the ALB5 we need to lay the groundwork for the necessary frame transformation calculations. This requires us to take measurements of the robot links. To do so we will need to declare some sort of "home position" so that we have baseline measurements. For the purpose of this lab we will declare our home position to be what is shown in Figure 4.

### 5.0.1 Measurements of Robotic Arm:

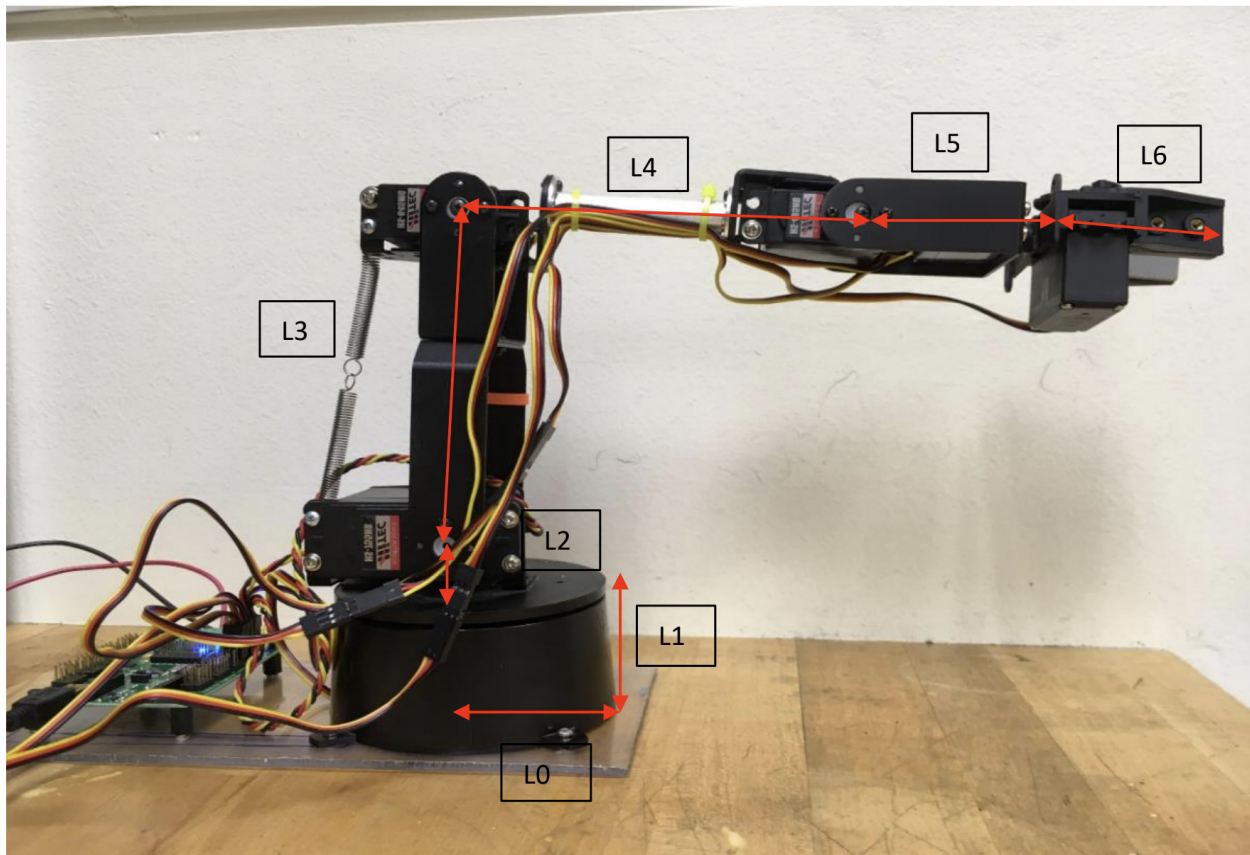


Figure 4: Home position robot configuration

- To achieve our "home position" as seen in Figure 4 we need to connect our arm to a power source as well as USB for communication.
- There should be two lead wires connected to the screw terminal of the SSC-32 board. These are our power and ground wires.
- When connecting power to the SSC-32 board make sure the output of the power supply is OFF. Set the supply voltage to 5V and current limit to at-least 1.8A. Now connect the red lead to red wire, and black lead to black wire and turn ON the power supply.

- Your SSC-32 board should now have power. The next step is to make sure ROS can communicate with the SSC-32 board to send out servo positions to the ALB5.
- Within your Ubuntu 20.04 environment create a workspace that you would like to use for the ALB5. You are welcome to call it whatever you like, but for the purposes of this lab it will be declared as follows (where fmlxxxx is your first, middle, and last initial and xxxx corresponds to your RIT ID):

Terminal

```
$ mkdir ~/fmlxxxx_alb5_ws/src
```

- Navigate to the /src folder of your newly created workspace:

Terminal

```
$ cd ~/fmlxxxx_alb5_ws/src
```

- Download the .zip file from myCourses and unzip it into your /src folder.
- Now navigate back to the root directory of your workspace (/fmlxxxx\_alb5\_ws) and `catkin_make` the folder.
- Source the new setup file:

Terminal

```
$ source devel/setup.bash
```

- Start a roscore:

Terminal

```
$ roscore
```

- Give sudo permissions to the USB device (you may need to unplug the USB from your laptop and replug it back in. Before running this command a popup should occur inside your VM asking you if you'd like the USB device to connect to your host machine or virtual machine, click virtual machine and then ubuntu 20.04):

Terminal

```
$ sudo chmod 777 /dev/ttyUSB0
```

- Now run the launch file, this will send a slew of commands to the SSC-32 board to control the servos, as well as launch a visualization in RViz so you can see what ROS thinks the position of the ALB5 arm is:

**Terminal**

```
$ roslaunch alb5_description display.launch
```

- The main point of this package is the `ssc32_node`. This will take a `.config` file detailing the joints to be controlled, the channels they're on, the range of the joints, etc. There is an example `.config` file in the `config` folder in the package.

**.config File**

```
baud: 115200
port: /dev/ttyUSB0
controllers: {
  joint_controller_1: {
    type: joint_controller,
    joints: [joint_1, joint_2, joint_3],
    publish_joint_states: true,
    publish_rate: 10
  },
  joint_controller_2: {
    type: joint_controller,
    joints: [joint_4, joint_5],
    publish_joint_states: true,
    publish_rate: 10
  }
}
joints: {
  joint_1: {channel: 0, max_angle: 1.57, min_angle: -1.57, offset_angle: 0.0,
    default_angle: 0.0, initialize: true, invert: false},
  joint_2: {channel: 1, max_angle: 1.57, min_angle: -1.57, offset_angle: 0.0,
    default_angle: 0.0, initialize: true, invert: false},
  joint_3: {channel: 2, max_angle: 1.57, min_angle: -1.57, offset_angle: 0.0,
    default_angle: 0.0, initialize: true, invert: false},
  joint_4: {channel: 3, max_angle: 1.57, min_angle: -1.57, offset_angle: 0.0,
    default_angle: 0.0, initialize: true, invert: false},
  joint_5: {channel: 4, max_angle: 1.57, min_angle: -1.57, offset_angle: 0.0,
    default_angle: 0.0, initialize: true, invert: false}
}
```

- The `.config` file takes the baud rate and the port. The controllers indicate a grouping of joints; `joint_controller_1` controls three joints, while `joint_controller_2` controls only 2. Both controllers will publish the joint states, at 10 Hz. The joints are also listed with names and specifics. The `.config` file will be loaded in as parameters. Information about them can be found [HERE](#).

- Once you `roslaunch` the launch file RViz will pop up along with sliders for each of the joints we can publish to. By default all of the joints are set to 0 radians, but this may not achieve the desired home position.
- Use the sliders to achieve the desired home position as shown in Figure 4. Make sure to record these numbers and add them to your `.config` file so that upon relaunching your launch file your ALB5 arm will be in its corrected home position.
  - Edit the `offset_angle` in the `alb5_ssc32.launch` launch file to achieve this behavior.
- Use the slider GUI to identify good values for the home position and add it to the `.config` file. Run the necessary launch file and demo to the TA for check off. Additionally include these numbers in a table as shown in Table 1

Table 1: Robot Servo Values and Degrees

| Servo/Joint | Center Value | Angle of Freedom |            |
|-------------|--------------|------------------|------------|
|             |              | Min Degree       | Max Degree |
| Base        |              |                  |            |
| Shoulder    |              |                  |            |
| Elbow       |              |                  |            |
| Wrist       |              |                  |            |
| Wrist Twist |              |                  |            |
| Gripper     |              |                  |            |

- Using the information given in Figure 4, determine the lengths (cm) of the listed links:

Table 2: Link Lengths

|    | Length (cm) |
|----|-------------|
| L0 |             |
| L1 |             |
| L2 |             |
| L3 |             |
| L4 |             |
| L5 |             |
| L6 |             |

### 5.0.2 Understanding URDF

- We will use a Universal Robot Description Format file (URDF). One has been made for the ALB5 robot. URDFs are XML files that describe a specific robot. They follow

a tree structure, beginning with a base link and child links, connecting to each other with joints.

- You can find the `ALB5.urdf` under your `/src/alb5_description/urdf` folder. It starts with the name of the robot, `alb5`. It then shows the first link, `universal_base`. A link typically has the properties `inertial`, `visual`, and `collision`. `Inertial` describes the inertia of the link, `visual` describes how the link looks, and `collision` describes the physical shape of the link. The `origin` property in all three indicates the reference frame. The `geometry` property indicates the shape, and can be indicated by a generic shape, like a cylinder, or a `.STL` mesh.
- The URDF then lists a second link, `base_link`, and the joint that connects the two links, `base`. Joints will have a type (revolute in this case), origin, parent and child link, the axis it moves about, dimension limits, and dynamics.
- URDFs typically come in a package labeled `<my_robot>_description`. These packages don't have executable code, but has the URDF, any meshes, and possibly config and launch files. URDFs themselves can be coded by hand, or autogenerated.

### 5.0.3 Publisher

- Using the provided `joint_move.py` write a publisher to move all robot joints linearly in incremental theta values for the joints

### 5.0.4 Repeatability test

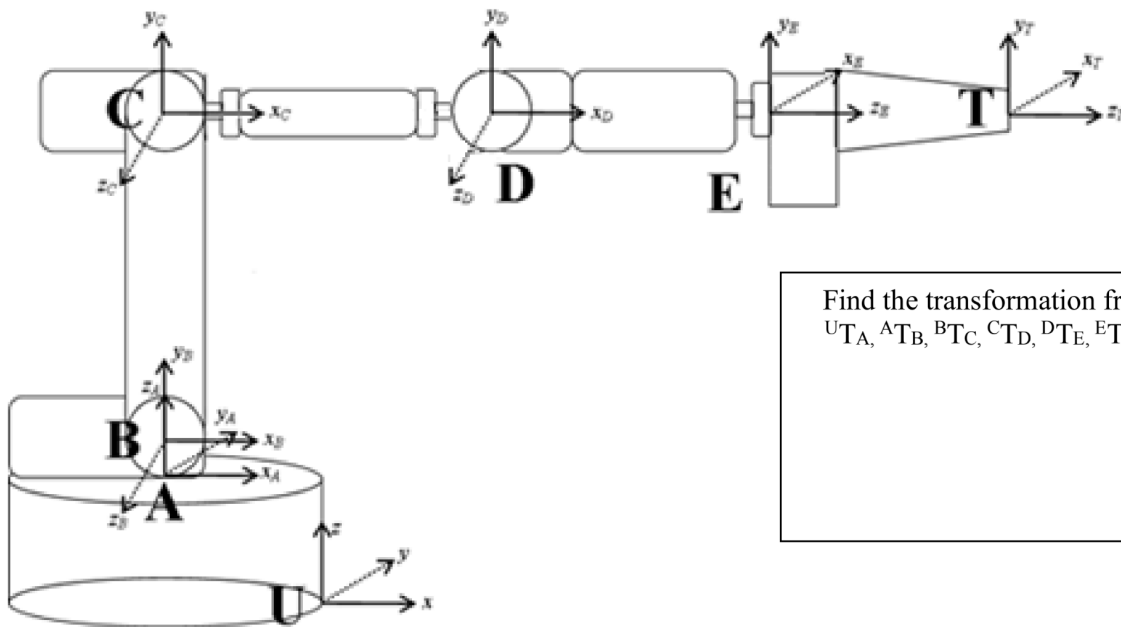
In this test you will write a simple publisher script and publish to the joint state topics to make the robot do a simple movement.

- **Step 1:** Move the robot to its home position with a thin pencil in its grip.
- **Step 2:** Set the robot's speed moves slower. (Increase the resolution of the increments)
- **Step 3:** Move the tip of the robot (the pencil) to a position approximately 12cm away from the square base by moving shoulder, elbow and wrist joints. Make sure that the marker is touching to paper straight to create a data point.
- **Step 5:** Repeat these steps 9 more times and then compute the radius of repeatability from the results.
- **Step 6: Give a demo,** include the Movement file and results in your report.
- **Step 7:** Repeat the experiment by also moving the base joint (make sure the pen is vertical at the contact position). **Give a demo,** include the Movement file and results in your report.

### 5.0.5 Simple Pick and Place

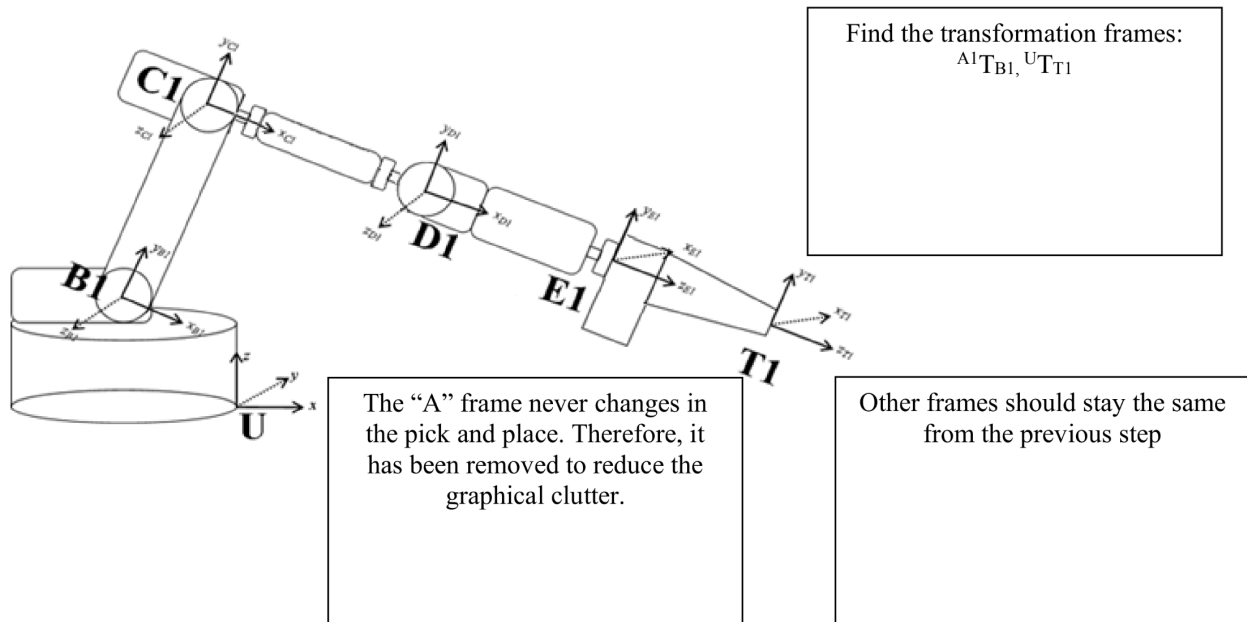
Create a pick and place routine as shown below, make sure that you include all frame transformation calculations. You **MUST** use the frame transformations to deduce the angle you need to achieve the desired position. Show the routine to the TA to be checked off.

- STEP 0: Home position

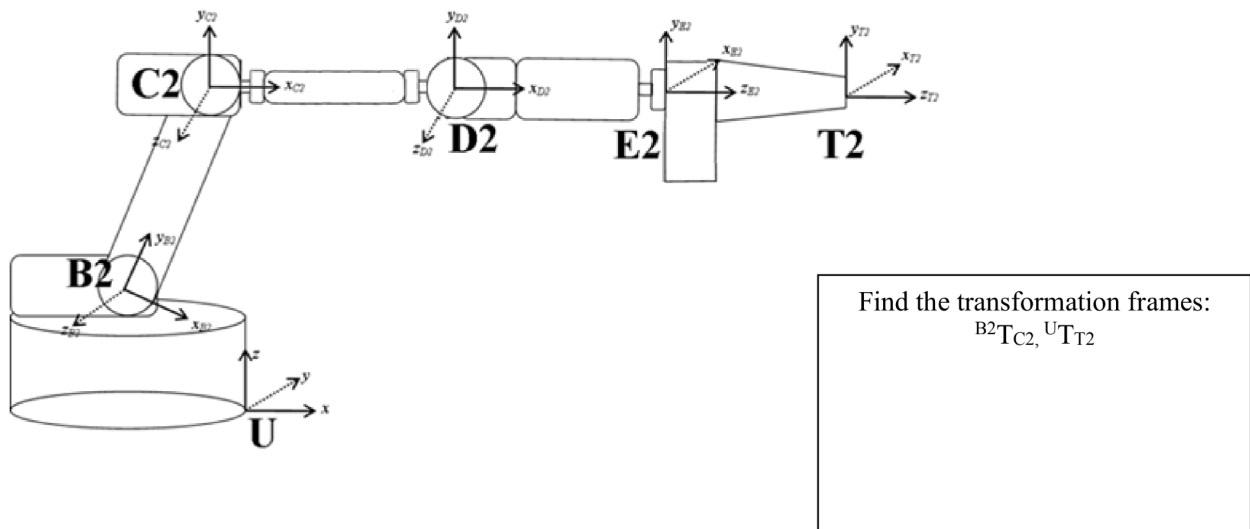


Find the transformation frames:  
 $U_{TA}$ ,  $A_{TB}$ ,  $B_{TC}$ ,  $C_{TD}$ ,  $D_{TE}$ ,  $E_{TT}$ ,  $U_{TT}$

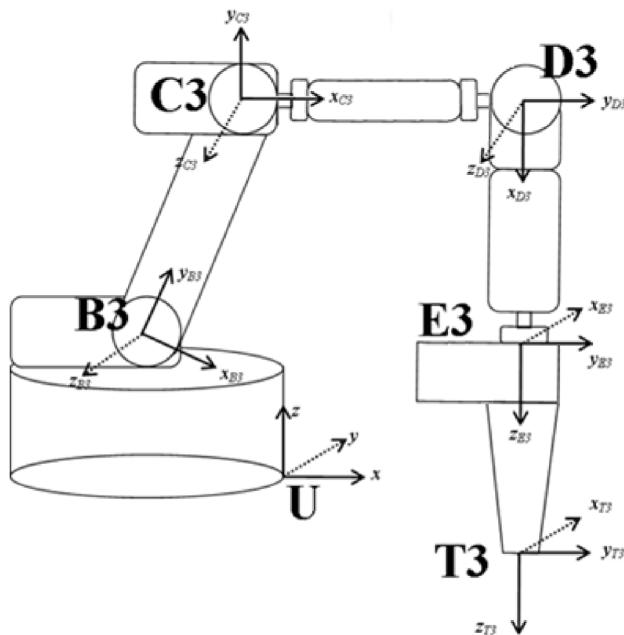
- STEP 1: Rotate shoulder servo about the  $Z_B$  axis by  $-30^\circ$ .



- STEP 2: Rotate elbow servo about the  $Z_{C1}$  axis by  $30^\circ$ .



- STEP 3: Rotate wrist servo about  $Z_{D2}$  axis by  $-90^\circ$ .



Find the transformation frames:  
 ${}^{C3}T_{D3}$ ,  ${}^UT_{T3}$

- STEP 4: Close the gripper (No effect on Tool Frame orientation/position).
- STEP 5: Open the gripper (No effect on Tool Frame orientation/position)

### 5.0.6 Forward Kinematics

Given end point coordinates and a set of (three) joint angles, the student will have to do the forward kinematic calculations and select the correct joint configuration for the end point coordinate.

## 6 Deliverables

All deliverables for this lab have been boldfaced, additionally they are listed below with links to them in the lab document. All deliverables are required to be in your report. This means that any tables you create, any code you write, or any calculations you make need to be in the report or your risk losing points.

1. Demo the correct home position joint angles, and your **.config** file as shown here: [5.0.1](#)
2. Demo your publisher as it increases all joint values incrementally as shown here: [5.0.3](#)
3. Demo the repeatability test so as to show the robot can effectively draw a circle as described here: [5.0.4](#)
4. Demo your simple pick and place with frame transformations as shown here: [5.0.6](#)

## 7 Report

- Submit a tech memo. Specific items to this lab that should be included are:
  - Link and Joint measurements.
  - Repeatability Discussion and results.
  - Output of frame transformations (calculations).
  - RQT plot of ALB5's Nodes
- Submission is to be done electronically using mycourses.
- You must submit a PDF or Microsoft Word compatible report (.pdf or .doc or .docx).
- You must follow the following dropbox submission guidelines:
  - Your tech memo document must be labeled "lastname\_585-685\_lab2"
- Late labs will receive a 10% late penalty per week late. The first 10% will be applied at 12:01am the day after the report/demos are due.

### Questions

- What is the arm robot lacking which would make it an easier robot to work with?
- What are the advantages and disadvantages of the arm robot over prismatic jointed robots?
- What were your thoughts of this lab, what could be improved, what did you like?

## Additional Reading Materials

Drake is an open-source robotics simulation and control toolbox developed by the Toyota Research Institute and maintained by the Robot Locomotion Group at MIT. It provides a flexible and powerful framework for modeling, simulating, and controlling robots in complex environments. Written primarily in C++ with Python bindings, Drake is designed for high-performance applications in robotics research and development. Drake supports dynamic simulation of multibody systems, automatic differentiation, optimization, and advanced control strategies. It includes a sophisticated geometry engine for collision detection and visualization, and integrates with tools like MeshCat and RViz for rendering and debugging. Its modular design enables users to build systems using interconnected components—such as sensors, actuators, and controllers—making it ideal for developing everything from simple robotic arms to full humanoid robots. One of Drake’s key strengths is its focus on realism and fidelity. It handles contact dynamics and friction accurately, which is essential for tasks like walking, grasping, and manipulation. Whether you’re prototyping new control algorithms or testing robotic designs in simulation before deployment, Drake offers a robust, research-grade foundation for modern robotics. It is accompanied by online textbooks that cover a range of robotic tasks. Notably, there is a comprehensive textbook on robotic manipulation that delves deeply into topics such as hardware, perception, and control. Below are links to selected chapters that will complement and expand on the material you’ve covered in class.

**Chapter: Pick and Place**

**Chapter: Geometric Pose Estimation**